Agenda

Practical concepts

- Constraints
- Space complexity
- Arrays
  - Reversal array
  - Rotation — Amazon | Microsoft

Qu

$2^k = n$, find k?

$\log 2^k = \log n$

$k \log 2 = \log n$

$k = \frac{\log n}{\log 2} = \log_2 n$

$\log a^b = b \log a$

$\frac{\log a}{\log b} = \log_b a$

Qu

itr $= 4n^2 + 3n + 1$

Big O $= O(n^2)$

Importance of constraints

Qu

Problem description ✓

Constraints ✗

i/p ✓

Examples ✓

Time taken to execute:

C++ → 1 sec

java → 2/3 sec

Python → 10 sec

✗

Common machine

└ 1 sec — 10^8 itr max.

Scenario

$1 <= n <= 10^5$

Krishna — $O(n)$ — 10^5 itr max — Works

n itr

Siddharta — $O(n^2)$ — $> 10^8$ — Not work

$(10^5)^2 = 10^{10}$ — TLE

jayanta — $O(n \log n)$ — works

$10^5 \log 10^5$

$< 10^8$

Yash → Logic → Code

Space complexity of algorithm

Extra space needed to solve an algorithm.

```
int[] doSomething (int n.) {
    4 // int x = 8
    4 // int tmp = 10
    4·n // int tmp[n]
        if (x==8) {
            print("ok");
        }
    return tmp;
}
```

$4 + 4 + 4 \cdot n$

$8 + 4 \cdot n$

$O(n)$

```
int[] doSomething (int[] A) {
    ____
    ____
    ____
    ____
    return A;
}
```

SC: $O(1)$

$O(n)$

100% correct

$O(1)$

forcing the developer to create the array

```
fun(n) {
  int arr[10]      ———————— 4 · 10 = 40 units
   "   x           ———————— 4
   "   y           ———————— 4
  long z           ———————— 8
  int arr[n]       ———————— 4 · n
}
```

$$40 + 4 + 4 + 8 + 4n$$

$$56 + 4n = O(n)$$

```
fun(n) {
  int x = n        ———————— 4
  int y = x * x    ———————— 4
  long z = x + y   ———————— 8
  int arr[n]       ———————— 4n
  long l[n][n]     ———————— 8 · n^2
}
```

$$4 + 4 + 8 + 4n + 8n^2 = O(n^2)$$

Arrays

collection of similar data types

1	2	8	10

→ int[]

4.8	2.69	1.5	3.01

→ double[]

1	2	2.8	6	5.9

→ invalid

int

float/double

"ayush"	"Mohan"	'x'	'y'

strings char

→ invalid

string

char

A =

0	1	2	3	4
5	8	1	20	16

A[2] = 1 ——— O(1)

A[4] = 16 ——— O(1)

A[8] = idx out of bound exception

A[5] = " " " " "

A[n] → idx[0, n-1]

Qu Print all el of array.

```
void print ( int[] A) {
    n = A.length
    for(i = 0; i <= n-1; i++) {
        print( A[i]);
    }
}
```

TC: O(n)
SC: O(1)

A[5] =

0	1	2	3	4
5	-4	8	9	10

sum of 1st & 5th element?

A[0] + A[4]

Qu Given A[n]. Reverse entire array.

ip

1	2	3	4	5

op

5	4	3	2	1

Expectation: TC: O(n)
SC: O(1)

signature

```
void reverseEntireArray (int[] A) {
    n = A.length;
    i = 0;
    j = n-1;
    while( i < j ) {
        // swap
        tmp = A[i];
        A[i] = A[j];
        A[j] = tmp;
        i++;
        j--;
    }
}
```

TC: O(n)
SC: O(1)

Example:

Swapping

0	1	2	3	4	5	6	7	8
~~1~~ 9	~~2~~ 8	~~3~~ 7	~~4~~ 6	5	~~6~~ 4	~~7~~ 3	~~8~~ 2	~~9~~ 1
i	i	i	i	i, j	j	j	j	j

~~1~~ 8	~~2~~ 7	~~3~~ 6	~~4~~ 5	~~5~~ 4	~~6~~ 3	~~7~~ 2	~~8~~ 1
i	i	i	i, j	j, i	j	j	j

$i >= j$ ——— stop

Qu Given A[n], start & end indices.
Reverse array from start to end.

ip

0	1	2	3	4
1	2	3	4	5

start = 1
end = 3

op

1	4	3	2	5

```
void reverse( int[] A, int start, int end) {
    n = A.length;
    i = ~~0~~; start
    j = ~~n-1~~, end
    while( i < j ) {
        // swap
        tmp = A[i];
        A[i] = A[j];
        A[j] = tmp;
        i++;
        j--;
    }
}
```

TC: O(n)
SC: O(1)

Reverse entire array : reverse(A, 0, n-1)

Break: 10:38 – 10:45

Qu A[n] —— Given

Rotate array K times from right to left.

Expectation: O(n), O(1)

1	2	3	4	5

K = 1

5	1	2	3	4

K = 2

4	5	1	2	3

K = 3

3	4	5	1	2

K = 4

2	3	4	5	1

K = 5

1	2	3	4	5

K = 6

5	1	2	3	4

K = 7

4 5 1 2 3

K = 8

3 4 5 1 2

K = 9

2 3 4 5 1

K = 10

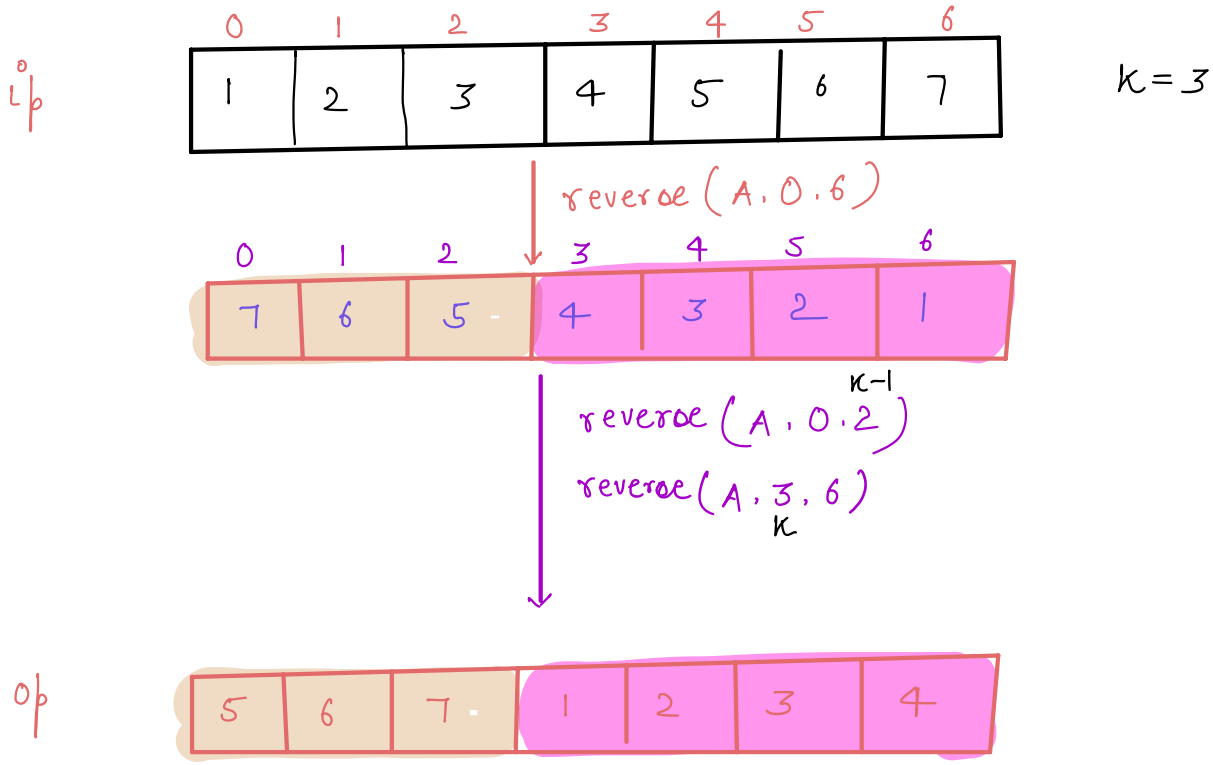
1 2 3 4 5

K = 11

5 1 2 3 4

```
void rotate (A[], K) {
```

K = 7 % 5 = 2




```
void rotate(A[], k) {
    n = A.length
    k = k % n;
    reverse(A, 0, n-1);  ———— O(n)  O(1)
    reverse(A, 0, k-1);  ———— O(n)  O(1)
    reverse(A, k, n-1);  ———— O(n)  O(1)
}
```

TC: O(n)

SC: O(1)

<u>Dry run</u> k = 5

	1	2	3	4	5
→	k = 0				
1)	5	4	3	2	1
2)	5	4	3	2	1
3)	1	2	3	4	5

Dynamic Arrays

Limitation of Array: fixed size.

overcome

Dynamic Arrays

- java: ArrayList
- C++: vector

```
int A[5];
```

A[0] = 5
A[1] = 10
A[2] = 18
A[3] = 1
A[4] = 18
~~A[5] = 20~~

can't add more el than size

```
List<Integer> list;
list.add(1);
list.add(2);
"    " (3)
"    " (4)
"    " (5)
"    " (6)
⋮
```

Thankyou ☺